Exported from http://10.90.105.221:8080/ on 21 Sep 2026 11:55 UTC

VM inventory — 7 VM(s) tracked, 0 Compliant. Last refreshed: 21 Sep 2026 11:55 UTC.

Windows Patch Automation Framework

Patch Windows VMs on VMware vCenter without RDP or WinRM. The runner connects to vCenter only; VMware Guest Operations runs update scripts inside each guest using the native Windows Update COM API.

Guest Operations only PowerCLI + PowerShell 7 JSONL audit logs Sep 15, 2026 · v1.2

End-to-end flow

From operator action to compliance report — every VM is an isolated worker with its own vCenter session.

End-to-end flow: Validate environment, connect to vCenter, copy scripts into the guest, repair Windows Update if needed, run patch cycles (scan, install, reboot), verify convergence, then export compliance reports.

Happy path in three steps

1. **Validate** — vCenter login works; VM exists; guest account works via a test `hostname` command.
2. **Patch loop** — Scan, download, and install all pending updates; reboot when required; repeat until zero pending.
3. **Report** — Compliance JSON/CSV/HTML plus per-VM logs under `output/<run-id>/`.

Quick start

Prerequisites

- VMware Tools running on target VMs (`guestToolsRunning`)
- Two credential pairs: vCenter SSO + Windows local admin inside each VM
- PowerCLI 13.3 and PowerShell 7.2+ on the runner
- Guest can reach WSUS or Microsoft Update

Run locally (fleet)

```
cd /home/tpx-admin/.cursor-server/Windows-Patch-Automation-Framework unset VCENTER_USERNAME
VCENTER_PASSWORD WINDOWS_GUEST_USERNAME WINDOWS_GUEST_PASSWORD source ~/.windows_patch_creds #
Validate only (enabled rows in config/vms.csv) pwsh -File
./scripts/Validate-PatchEnvironment.ps1 # Patch all enabled VMs in config/vms.csv (parallel,
throttled) ./scripts/execute_patch.sh
```

Inventory row

Set `Enabled=true` in `config/vms.csv` only for approved VMs. Use exact vCenter VM name or provide IP for fallback lookup.

Execution commands

Run from the Linux runner (`blr-kiruthika`). Credentials are loaded with `source` — never execute the cred file directly.

1. Credential file

Create one file per guest password group (same password can share one file):

```
vi ~/.windows_patch_creds chmod 600 ~/.windows_patch_creds
```

```
export VCENTER_USERNAME="ks4@vsphere.local" export VCENTER_PASSWORD="your-vcenter-password"
export WINDOWS_GUEST_USERNAME="admin" export WINDOWS_GUEST_PASSWORD="your-guest-password"
```

Example: `admin` for NEPTUNE/damo VMs; `Administrator` for DHA — use separate files if passwords differ:

```
~/.windows_patch_creds # damo / admin VMs ~/.windows_patch_creds_dha # BLR-DHA-TESTING02 /
Administrator
```

2. Single-VM inventory CSV

Create `output/<vm>-only.csv` so you do not patch the whole fleet:

```
VMName,Enabled,VCenterServer,Environment,Owner,MaintenanceWindow,IPAddress,FQDN,Notes
blr-damo-test02,true,blr-vsphere-01.strykercorp.com,BLR,Owner,Sunday-0200-UTC,10.90.120.242,blr-damo-test02
VM run
BLR-DHA-TESTING02,true,blr-vsphere-01.strykercorp.com,BLR,Geetha,Sunday-0200-UTC,10.90.105.67,BLR-DHA-TESTI
testing
```

3. Validate one server (read-only)

```
cd /home/tpx-admin/.cursor-server/Windows-Patch-Automation-Framework unset VCENTER_USERNAME
VCENTER_PASSWORD WINDOWS_GUEST_USERNAME WINDOWS_GUEST_PASSWORD source ~/.windows_patch_creds
pwsh -File ./scripts/Validate-PatchEnvironment.ps1 \ -InventoryPath
./output/blr-damo-test02-only.csv
```

Expect: **VM found**, **GuestHost=**, **Validation completed successfully**.

4. Patch one server

```
# Default cred file (~/.windows_patch_creds) ./scripts/run_patch_with_creds.sh
./output/blr-damo-test02-only.csv # Alternate cred file (e.g. DHA Administrator)
./scripts/run_patch_with_creds.sh \ ./output/blr-dha-testing02-only.csv \
~/.windows_patch_creds_dha
```

Equivalent manual form:

```
source ~/.windows_patch_creds_dha ./scripts/execute_repair_and_patch.sh
./output/blr-dha-testing02-only.csv
```

5. Monitor progress

```
ls -lt output/ | head -5 # Live orchestrator log (replace run-id and vm folder) tail -f
output/<run-id>/vm-logs/blr-damo-test02/orchestrator.jsonl # Final status cat
output/<run-id>/compliance-report.csv
```

Success: status `Compliant`, zero pending updates. Typical runtime: 10–30+ minutes.

When the patch finishes, the VM inventory section updates automatically (refresh the browser with Ctrl+F5).

Common mistakes

Problem	Fix
`Permission denied` on cred file	Use `source ~/.windows_patch_creds`, not `~/.windows_patch_creds`
Wrong guest user in validation output	`unset` env vars, then `source` cred file again
Guest auth failed	Match RDP/console username (`admin` vs `Administrator`)
Two patch jobs on same VM	Run one job at a time per VM

6. Patch entire fleet (`config/vms.csv`)

See Fleet run (all VMs) for the full checklist. Short form:

```
unset VCENTER_USERNAME VCENTER_PASSWORD WINDOWS_GUEST_USERNAME WINDOWS_GUEST_PASSWORD source
~/.windows_patch_creds ./scripts/execute_patch.sh
```

All enabled rows in `config/vms.csv` must share the **same guest username and password** in that run.

Fleet run (all VMs at once)

Use this when you want to patch every server listed in the main inventory (`config/vms.csv`) — the same list used for scheduled GitHub Actions.

Before you start

- Every VM you enable must use the **same** guest account for that run (`admin` or `Administrator`, one password).
- If passwords differ, split into batches (separate CSV + cred file per batch) — do not mix in one run.
- Take vCenter snapshots on production VMs before a fleet patch.
- Confirm VMware Tools are running on all targets.

Step 1 — Review inventory

```
cd /home/tpx-admin/.cursor-server/Windows-Patch-Automation-Framework # Show rows that will be
patched awk -F, 'NR==1 || $2 ~ /^(true|yes|1)$/i {print}' config/vms.csv
```

Edit `config/vms.csv` (or export from Excel and save as CSV with the same columns). Set `Enabled=true` only for VMs approved this window.

Step 2 — Credentials

```
vi ~/.windows_patch_creds chmod 600 ~/.windows_patch_creds unset VCENTER_USERNAME
VCENTER_PASSWORD WINDOWS_GUEST_USERNAME WINDOWS_GUEST_PASSWORD source ~/.windows_patch_creds
```

Step 3 — Validate fleet (read-only)

```
pwsh -File ./scripts/Validate-PatchEnvironment.ps1 \ -InventoryPath ./config/vms.csv
```

Fix any VM not found, Tools not running, or guest auth failures before patching.

Step 4 — Patch all enabled VMs

```
./scripts/execute_patch.sh
```

Equivalent PowerShell form:

```
pwsh -File ./scripts/Validate-PatchEnvironment.ps1 \ -InventoryPath ./config/vms.csv \
-ExecutePatch
```

Runs up to `throttleLimit` VMs in parallel (default 4 in `config/settings.json`).

Step 5 — Review reports

```
ls -lt output/ | head -5 cat output/<run-id>/compliance-report.csv # Open in browser:
output/<run-id>/compliance-report.html
```

Success: every row shows `Compliant` with zero pending updates.

Different guest passwords in the fleet

Example: NEPTUNE VMs use `admin`, DHA uses `Administrator`. Run two batches:

```
# Batch A — admin password group source ~/.windows_patch_creds ./scripts/run_patch_with_creds.sh
./output/neptune-batch-only.csv # Batch B — Administrator password group
./scripts/run_patch_with_creds.sh ./output/dha-batch-only.csv ~/.windows_patch_creds_dha
```

Create each batch CSV with only the VMs that share credentials, all with `Enabled=true`.

Adding a new machine

Onboard one server safely before enabling it in the main fleet CSV.

Collect before you add

Field	Example
Exact vCenter VM name	`blr-damo-test02`
IP (from vCenter guest summary)	`10.90.120.242`
vCenter	`blr-vsphere-01.strykercorp.com`
Guest username	`admin` or `Administrator`
Guest password	Must work via Guest Operations (not RDP-only)

Step 1 — Snapshot in vCenter

VM → Snapshots → Take Snapshot (before first patch).

Step 2 — Single-VM inventory file

```
cat > output/new-server-only.csv <<'EOF'
VMName,Enabled,VCenterServer,Environment,Owner,MaintenanceWindow,IPAddress,FQDN,Notes
YOUR-VM-NAME,true,blr-vsphere-01.strykercorp.com,BLR,OwnerName,Sunday-0200-UTC,10.90.x.x,your-vm.vcraeng.co
patch EOF
```

Do not put passwords in the CSV.

Step 3 — Credentials

```
unset VCENTER_USERNAME VCENTER_PASSWORD WINDOWS_GUEST_USERNAME WINDOWS_GUEST_PASSWORD source
~/.windows_patch_creds # or ~/.windows_patch_creds_dha
```

Step 4 — Validate (read-only)

```
pwsh -File ./scripts/Validate-PatchEnvironment.ps1 \ -InventoryPath ./output/new-server-only.csv
```

Expect: VM found, Tools running, `GuestHost=` shows hostname.

Step 5 — Patch the new server

```
./scripts/run_patch_with_creds.sh ./output/new-server-only.csv # or with alternate cred file:
./scripts/run_patch_with_creds.sh ./output/new-server-only.csv ~/.windows_patch_creds_dha
```

Step 6 — Confirm Compliant

```
cat output/<run-id>/compliance-report.csv
```

Step 7 — Add to main inventory

After a successful run, append the row to `config/vms.csv` with `Enabled=true` so it is included in fleet runs and GitHub Actions:

```
VMName,Enabled,VCenterServer,Environment,Owner,MaintenanceWindow,IPAddress,FQDN,Notes
YOUR-VM-NAME,true,blr-vsphere-01.strykercorp.com,BLR,OwnerName,Sunday-0200-UTC,10.90.x.x,your-vm.vcraeng.co
- first patch Sep 2026
```

Commit and push `config/vms.csv` if you use the GitHub pipeline.

Quick checklist

1. Snapshot VM
2. Create `output/<vm>-only.csv`
3. Set creds in `~/.windows_patch_creds`
4. Validate
5. Patch
6. Confirm Compliant
7. Add row to `config/vms.csv`

Full guide: `docs/New-Server-Execution-Steps.md` in the repo.

GitHub pipeline (Actions)

Yes — you can trigger patching from the GitHub repo. Workflow file: `.github/workflows/windows-patch.yml`.

What runs

1. **Validate** — Pester tests on GitHub-hosted Ubuntu (no vCenter needed).
2. **Patch** — Full fleet patch on a **self-hosted Windows runner** with labels `self-hosted`, `windows`, `vcenter`.
3. **Artifacts** — Uploads `output/` (reports + JSONL logs) for 30 days.

Prerequisites (one-time)

- Self-hosted Windows runner registered to the repo with labels above and network access to vCenter (HTTPS 443).
- GitHub environment `windows-patching` with secrets: `VCENTER_USERNAME`, `VCENTER_PASSWORD`, `WINDOWS_GUEST_USERNAME`, `WINDOWS_GUEST_PASSWORD`.
- Target VMs listed in `config/vms.csv` with `Enabled=true`.
- One guest password per workflow run — batch only VMs that share the same guest credentials.

Manual trigger from GitHub UI

1. Open Actions → Windows Patch Automation.
2. Click **Run workflow** (dropdown on the right).
3. Select branch `main`, then **Run workflow**.
4. Wait for job *Validate framework*, then *Patch Windows fleet*.
5. Download artifact `windows-patch-results-<run-id>` when complete.

Trigger from CLI (gh)

```
gh workflow run windows-patch.yml --repo Kiruthika-syk/Windows-Patch-Automation-Framework #
Watch run status gh run list --workflow=windows-patch.yml --repo
Kiruthika-syk/Windows-Patch-Automation-Framework gh run watch --repo
Kiruthika-syk/Windows-Patch-Automation-Framework
```

Scheduled runs

Cron is configured for **Sunday 02:00 UTC** (`0 2 * * 0`). Edit the workflow file to change the schedule.

GitHub vs Linux runner

GitHub Actions patches all enabled VMs in `config/vms.csv` using repo secrets.

Linux runner (blr-kiruthika) is better for single-VM runs via `output/*-only.csv` and per-server cred files — what you use for damo/DHA testing today.

Connection mechanism

Two separate credential pairs — never mix them up.

Environment variable	Used for	Example
`VCENTER_USERNAME` / `VCENTER_PASSWORD`	vCenter SSO login	`ks4@vsphere.local`
`WINDOWS_GUEST_USERNAME` / `WINDOWS_GUEST_PASSWORD`	Inside each VM via Guest Ops	`admin` or `Administrator`

Network path

Runner ■■HTTPS 443■■■ vCenter (e.g. blr-vsphere-01.strykercorp.com) ■ ■■■ VMware Tools ■■■ Windows guest Invoke-VMScript · Copy-VMGuestFile

- Runner does **not** use RDP (3389), WinRM (5985), or SMB to the VM IP.
- Guest downloads updates via its normal Windows Update / WSUS path.

Key PowerCLI operations

Operation	Purpose

`Connect-VIServer`	Authenticate to vCenter
`Get-VM` + IP fallback	Resolve exact VM object
`Copy-VMGuestFile -LocalToGuest`	Upload patch scripts to guest folder
`Invoke-VMScript`	Run PowerShell inside the guest as local admin
`Copy-VMGuestFile -GuestToLocal`	Download result JSON and logs
`Restart-Computer` via Invoke-VMScript	Reboot guest; wait for Tools to return

Scripts reference

One-line role for each component in the framework.

run_patch_with_creds.sh

Patch one inventory CSV using a chosen credential file.

Update-DocsSite.ps1

Rebuilds `docs/fleet-status.json` for the internal website (runs automatically after each patch).

execute_repair_and_patch.sh

Loads credentials, validates, then runs full patch job.

Validate-PatchEnvironment.ps1

Read-only: tests vCenter + guest login per VM.

Invoke-WindowsPatchAutomation.ps1

Main orchestrator: parallel workers, cycles, reboots, reports.

VCenter.GuestOps.psm1

Connect, VM lookup, script staging, polling, reboot, convergence.

Framework.Common.psm1

Config validation, JSONL logging, retries, report export.

Guest/Invoke-WindowsUpdate.ps1

Inside VM: one scan / download / install cycle via WU COM.

Guest/Repair-WindowsUpdateAgent.ps1

Inside VM: fix broken WU COM (services, cache, DISM/SFC).

Resolve-VMInventory.ps1

Maps spreadsheet IPs to exact vCenter VM names.

Update checking process

Runs inside the guest via `Microsoft.Update.Session` COM API.

1. **Warm-up** — After recent reboot, start WU services and wait 45–90 seconds.
2. **Primary scan** — `IsInstalled=0 and IsHidden=0`
3. **Definition scan** — Same criteria plus `CategoryIDs contains '798'` (Defender / definition updates).
4. **Merge** — Combine results so security intelligence updates are not missed.
5. **Evaluate** — Zero updates + no registry reboot pending → `NoUpdates` (done).

Reboot detection uses registry keys (CBS, WU RebootRequired), not only the COM `RebootRequired` flag — this prevents false reboot loops after updates are installed.

Update installation process

1. Accept EULAs for each applicable update.
2. Download all updates in one batch (`CreateUpdateDownloader`).
3. Install all downloaded updates in one batch (`CreateUpdateInstaller`).
4. Record per-update title, KB, result code, HRESULT, reboot flag.
5. Write `cycle-N-result.json` atomically; runner polls for this file.

If COM fails with `0x800703FA` shortly after reboot, the guest waits and retries before renaming the SoftwareDistribution cache.

Orchestrator patch loop

For each enabled row in `config/vms.csv`, up to `throttleLimit` VMs run in parallel.

Phase	What happens
Connect	Connect-PatchVCenter with exponential retry
Resolve VM	By name; fallback match by IP from inventory
Tools ready	Poll until guestToolsRunning and guest state = running
Stage	Copy patch + repair scripts to guest working directory
Repair	Quick COM test; basic repair; DISM/SFC only if still broken
Cycle 1..N	Start async guest job → poll → copy artifacts

Reboot	Restart-Computer → wait Tools → 90s service warm-up
Convergence	0 pending + no registry reboot → Compliant

Final status values

Status	Meaning
Compliant	No pending updates; convergence verified
MaxCyclesExceeded	Safety cap hit (updates may still have installed)
Failed	Hard error: COM, timeout, auth, etc.

Logging and reports

All audit data uses JSON Lines (one JSON object per line) plus structured JSON reports.

Output file	Written by	Contents
`master.jsonl`	Orchestrator	Merged run log
`vm-logs/<VM>/orchestrator.jsonl`	Orchestrator	Connect, cycles, reboots, errors
`cycle-N-guest.jsonl`	Guest script	Scan, download, install steps
`cycle-N-result.json`	Guest script	Counts, update list, status
`compliance-report.json`	Orchestrator	Final fleet result + update details
`compliance-report.csv / .html`	Orchestrator	Summary for Excel and browsers

Sample log entry

```
{ "timestampUtc": "2026-09-08T11:20:00.3209404Z", "level": "Information", "stage": "PatchCycle",
"vmName": "BLR-AARYA-VS", "message": "Starting patch cycle 1.", "data": {} }
```

Configuration

File	Purpose
`config/settings.json`	Timeouts, max cycles, warm-up, search criteria, throttle
`config/vms.csv`	VM inventory; only Enabled=true rows are patched
`config/vcenters.json`	BLR / FW / STC vCenter server names
`~/.windows_patch_creds`	Local credentials (not in git)

Key settings

Setting	Default	Role
`maxPatchCycles`	8	Max install/reboot cycles per VM
`maxRebootOnlyCycles`	2	Cap empty reboot loops
`postRebootWarmUpSeconds`	90	WU service settle time after reboot
`throttleLimit`	4	Parallel VM workers

VM inventory

All VMs from `config/vms.csv`, `output/*-only.csv`, and completed patch runs on `blr-vsphere-01.strykercorp.com`. **Auto-updated** after every patch job.

Last refreshed: 21 Sep 2026 11:55 UTC

0

Compliant

0

Patching

7

Total VMs

26.0

Updates installed

VMName	IP	Owner	FQDN	Enabled	Result	Updates	Cycles	Last run	Notes
blr-122105 VMP	10.90.122.105	NEPTUNE	blr-122105vmp.vraeng.com	True	Registered	0	0	Not started	Re-patch after MaxCyclesExceeded fix
blr-122106 VS	10.90.122.106	NEPTUNE	blr-122106vs.vraeng.com	True	Registered	0	0	Not started	Compliant - Azure Arc KB5121364 installed Sep 2026

BLR-AARYA-VS	10.90.128.199	Shrikant Raghuvanshi	blr-aarya-vs.vcraeng.com	Registered	0	0	Not started	Project Macaw - 6 pending updates
blr-astrion-01	10.90.129.16	EMS Engineering	blr-astrion-01.vcraeng.com	MaxCyclesExceeded	12	12	9cdf690e · 21 Sep 2026 06:25 UTC	Applicable updates remain after the safety cap of 12 cycles.
blr-b7-beta-vcg1	10.90.104.195	Nagendra Babu Gurram	blr-b7-beta-vcg1.vcraeng.com	Registered	0	0	Not started	EMS Dev Firmware - first patch
blr-damo-test02	10.90.120.242	Vijay Bhaskar	blr-damo-test02.vcraeng.com	Registered	0	0	Not started	Vocera Analytics Automation - INC0038523 Dev EMS
BLR-DHA-TESTING02	10.90.125.67	Geetha	BLR-DHA-TESTING02.vcraeng.com	Registered	0	0	Not started	DHA testing server - Devices Firmware EMS

Security

- Credentials live in environment variables or GitHub secrets — never in CSV, logs, or reports.
- Guest scripts run as local admin only through VMware Guest Operations (audited in vCenter).
- No inbound management ports required on VMs beyond VMware Tools and Windows Update egress.

More documentation

- Windows-Patch-Automation-Guide.pdf — Full website content as PDF (includes live VM inventory snapshot)
- Process-Flow-and-Mechanism.md — Markdown source (same content)
- Windows-Patch-Automation-Manual.md — Full operations manual
- TROUBLESHOOTING.md — Error diagnosis
- README.md — Repository overview

- GitHub repository